

Análisis de datos

Nombres: Josue Carranza Saucedo

Laura Isela Martínez Aguilera

Diana Laura Nava Rea

**Número de control: IS21110900, IS21110649,
IS21110681**

**Escuela: Instituto Tecnológico Superior de
Irapuato**

Semestre: 8

Equipo: 7

**Practica 2. Detección y corrección de
inconsistencias en datos.**

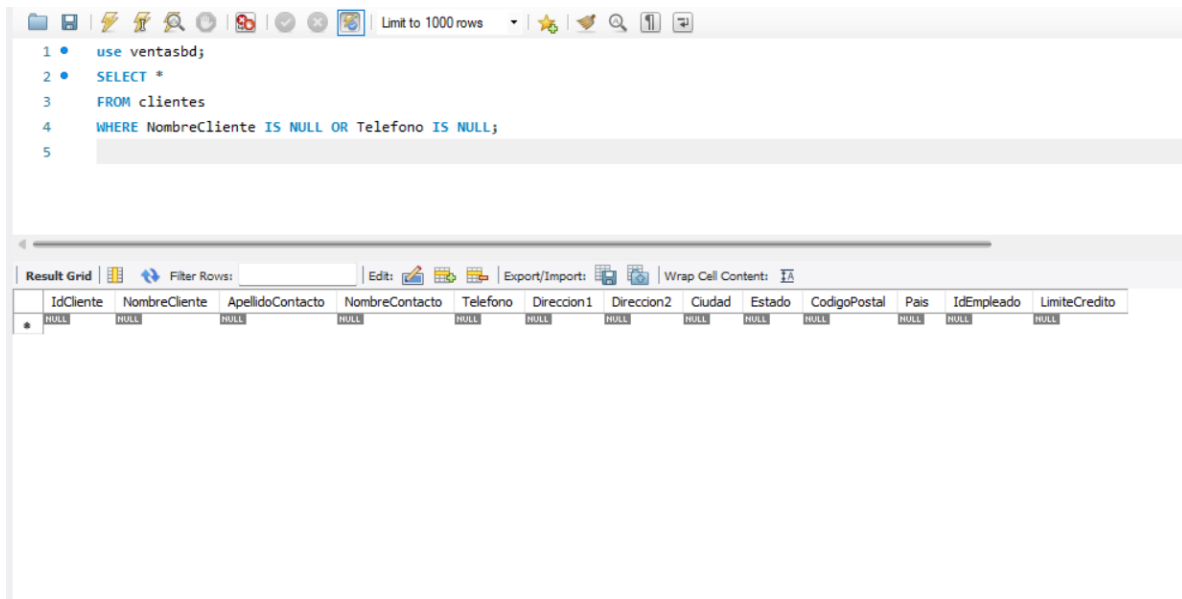
INTRODUCCIÓN

El objetivo de esta práctica es identificar registros nulos, duplicados o inconsistentes en la base datos y corregirlos. Realizamos múltiples consultas y sentencias en MySQL para trabajar sobre la base de datos de “ventasbd” que estuvimos empleando en la práctica pasada. En esta ocasión lo que se realizó fueron verificaciones, consultas, actualizaciones y eliminaciones de registros que estuvieran erróneos o duplicados. A continuación, se muestran las actividades junto a sus códigos empleados para ser realizadas y la evidencia de aplicación junto a los resultados de cada uno.

PRIMERA ACTIVIDAD

Hacer un query en la tabla clientes donde su nombre cliente o teléfono sea nulo.

Evidencia de aplicación y resultados obtenidos:



The screenshot shows a database query tool interface. The top toolbar includes icons for file operations, search, and execution. The SQL editor contains the following query:

```
1 • use ventasbd;  
2 • SELECT *  
3 FROM clientes  
4 WHERE NombreCliente IS NULL OR Telefono IS NULL;  
5
```

Below the query editor is a horizontal scrollbar. The bottom section displays the 'Result Grid' with a toolbar for editing and exporting. The grid shows a single row with all columns containing 'NULL' values, indicating no records were found.

IdCliente	NombreCliente	ApellidoContacto	NombreContacto	Telefono	Direccion1	Direccion2	Ciudad	Estado	CodigoPostal	Pais	IdEmpleado	LimiteCredito
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

En este caso, no hemos recibido ningún registro, puesto que no se encontraron Clientes con “NombreCliente o Teléfono nulos”

SEGUNDA ACTIVIDAD

Buscar y encontrar a través de un query los registros duplicados de la tabla cliente.

Evidencia de aplicación y resultados obtenidos:

```

6
7 -- SEGUNDA ACTIVIDAD --
8 • SELECT NombreCliente , Telefono, COUNT(*) AS total
9 FROM clientes
10 GROUP BY NombreCliente, Telefono
11 HAVING COUNT(*) > 1;
12
13 -- TERCERA ACTIVIDAD --
14
15
16
17

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

NombreCliente	Telefono	total
---------------	----------	-------

Result 2 x | Read Only

Output

#	Time	Action	Message	Duration / Fetch
1	20:47:32	SELECT NombreCliente , Telefono, COUNT(*) AS total FROM clientes GROUP BY NombreCliente, Telefono HA...	0 row(s) returned	0.000 sec / 0.000 sec

Como podemos visualizar, no hay registros duplicados de “NombreCliente” o “Telefono” en la tabla clientes, es por ello que se nos retorna una tabla vacia.

TERCER ACTIVIDAD

Identificar compras a través de un query con cantidad o precios negativos de la tabla detalles compra.

Evidencia de aplicación y resultados obtenidos:

```

13 -- TERCERA ACTIVIDAD --
14 • SELECT *
15 FROM detallescompra
16 WHERE CantidadOrdenada = 0 OR PrecioUnitario < 0;
17
18
19
20
21

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

IdCompra	IdProducto	CantidadOrdenada	PrecioUnitario	NumeroCompra
NULL	NULL	NULL	NULL	NULL

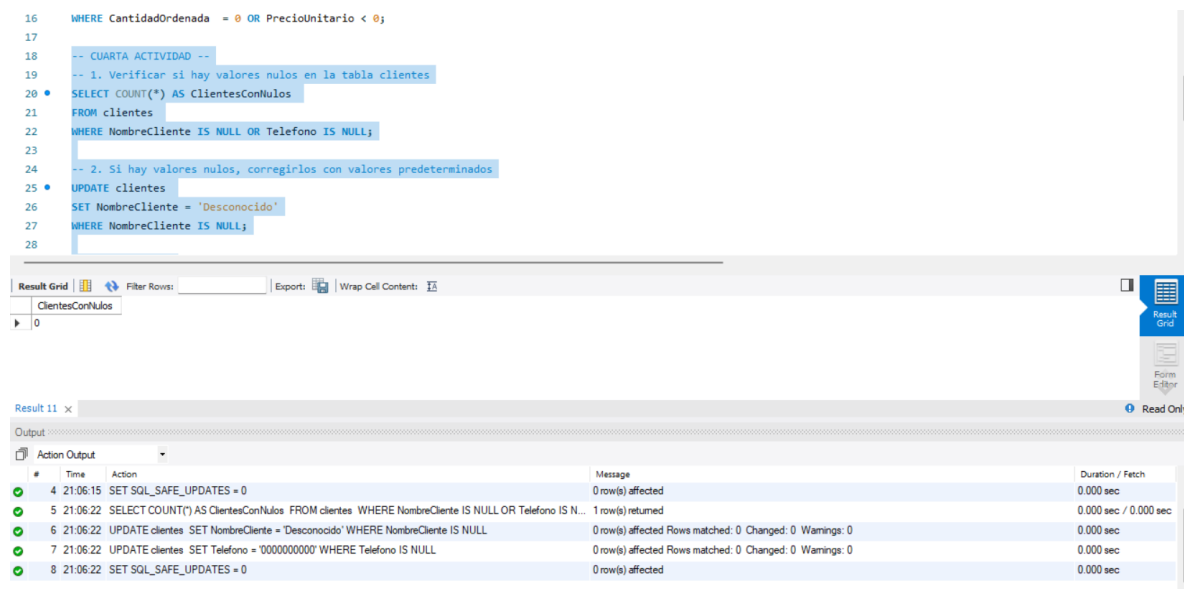
Como visualizamos, hemos aplicado el select en la tabla detallescompra en donde la Cantidad ordenada sea igual a 0 o que los precios unitarios

sean valores negativos, sin embargo, no se encontró ningún registro que cumpla las condiciones.

CUARTA ACTIVIDAD

Corregir datos con **UPDATE** si aplica o eliminar los registros incorrectos con **DELETE**.

Evidencia de aplicación y resultados obtenidos:



```
16 WHERE CantidadOrdenada = 0 OR PrecioUnitario < 0;
17
18 -- CUARTA ACTIVIDAD --
19 -- 1. Verificar si hay valores nulos en la tabla clientes
20 • SELECT COUNT(*) AS ClientesConNulos
21 FROM clientes
22 WHERE NombreCliente IS NULL OR Telefono IS NULL;
23
24 -- 2. Si hay valores nulos, corregirlos con valores predeterminados
25 • UPDATE clientes
26 SET NombreCliente = 'Desconocido'
27 WHERE NombreCliente IS NULL;
28
```

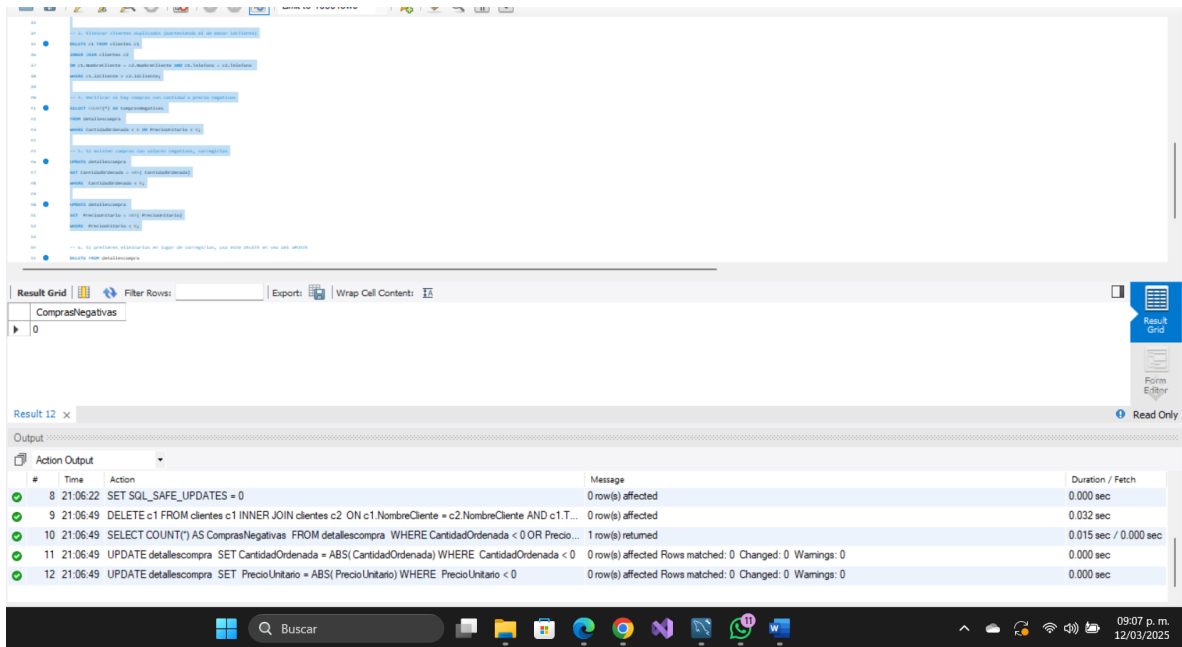
Result Grid

ClientesConNulos
0

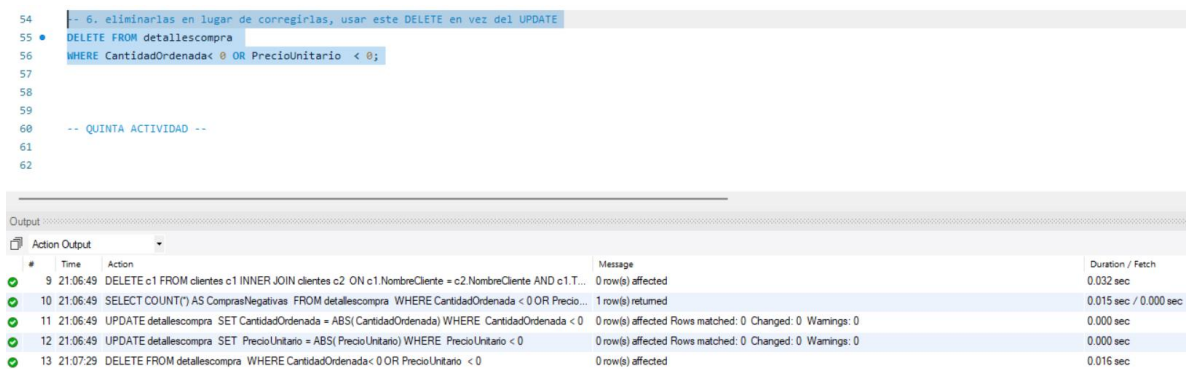
Output

#	Time	Action	Message	Duration / Fetch
4	21:06:15	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
5	21:06:22	SELECT COUNT(*) AS ClientesConNulos FROM clientes WHERE NombreCliente IS NULL OR Telefono IS N...	1 row(s) returned	0.000 sec / 0.000 sec
6	21:06:22	UPDATE clientes SET NombreCliente = 'Desconocido' WHERE NombreCliente IS NULL	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0	0.000 sec
7	21:06:22	UPDATE clientes SET Telefono = '0000000000' WHERE Telefono IS NULL	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0	0.000 sec
8	21:06:22	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec

En esta primera captura, verificamos si realmente hay valores nulos en nuestra tabla para comenzar a trabajar sobre ellos, y si es que, si hay, se corrigen con valores predeterminados.



En esta segunda captura ejecutamos la sentencia que eliminara a los clientes que se encuentren duplicados pero mantendrá a los que tengan el menor id, el que **probablemente** fue el primero en ser ingresado en la tabla (si es que Idcliente es un campo autoincremental, lo que es común), en la siguiente sentencia hacemos una verificación para saber si existen valores negativos en cantidad o precio y si es así en la sentencia posterior se corrigen empleando UPDATE tanto en la cantidad ordenada como en el precio unitario.



QUINTA ACTIVIDAD

Hacer análisis de los cambios realizados al maestro y la exposición en el equipo.

Evidencia de aplicación y resultados obtenidos:

The screenshot displays a SQL query execution interface. The query is as follows:

```
-- QUINTA ACTIVIDAD --
1
2 SELECT 'Clientes con datos nulos corregidos' AS Descripción, COUNT(*) AS RegistrosAfectados
3 FROM clientes WHERE NombreCliente = 'Desconocido' OR Telefono = '0000000000'
4 UNION ALL
5 SELECT 'Clientes duplicados eliminados', ROW_COUNT()
6 UNION ALL
7 SELECT 'Compras con cantidad negativa corregidas/eliminadas', COUNT(*)
8 FROM detallescompra WHERE CantidadOrdenada < 0
9 UNION ALL
10 SELECT 'Compras con precio negativo corregidas/eliminadas', COUNT(*)
11 FROM detallescompra WHERE PrecioUnitario < 0;
```

Below the query, the results are shown in a table:

Descripción	RegistrosAfectados
Clientes con datos nulos corregidos	0
Clientes duplicados eliminados	0
Compras con cantidad negativa corregidas/elimi...	0
Compras con precio negativo corregidas/elimina...	0

Below the results, the Action Output log is displayed:

#	Time	Action	Message	Duration / Fetch
10	21:06:49	SELECT COUNT(*) AS ComprasNegativas FROM detallescompra WHERE CantidadOrdenada < 0 OR Precio...	1 row(s) returned	0.015 sec / 0.000 sec
11	21:06:49	UPDATE detallescompra SET CantidadOrdenada = ABS(CantidadOrdenada) WHERE CantidadOrdenada < 0	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0	0.000 sec
12	21:06:49	UPDATE detallescompra SET PrecioUnitario = ABS(PrecioUnitario) WHERE PrecioUnitario < 0	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0	0.000 sec
13	21:07:29	DELETE FROM detallescompra WHERE CantidadOrdenada < 0 OR PrecioUnitario < 0	0 row(s) affected	0.016 sec
14	21:09:33	SELECT 'Clientes con datos nulos corregidos' AS Descripción, COUNT(*) AS RegistrosAfectados FROM client...	4 row(s) returned	0.016 sec / 0.000 sec

Se obtuvieron los resultados finales, en donde hicimos modificaciones o cambios a los clientes con algún registro nulo o duplicado, así como las compras con cantidad negativa y las compras con precio negativo. Por lo que se muestra en la consulta, ninguno fue modificado o cambiado porque no había alguno con datos inconsistentes o erróneos.

CONCLUSIONES

Esta práctica fungió para darnos una muestra de la detección y corrección de inconsistencias en los datos, lo cual nos permitió identificar las áreas en la base de datos empleada en donde se requería hacer modificaciones y correcciones en los registros que se encontraran en un estado “NULO” o duplicado. Es crucial tener la capacidad de poder detectar y dar solución a los datos que lo necesiten en tiempo y forma para que la consistencia de estos se mantenga y no interfiera con la integridad de la base de datos.